Classic + Simple

Guía de código de `TeleCon_Classic_Simple`

ESP32 DevKit · Bluetooth Classic · líneas de texto

Versión 1.0 2026

Abstract

Esta nota es el sketch de Arduino en la carpeta `Classic_Simple/TeleCon_Classic_Simple/`. El ajuste del teléfono es Classic + Simple. El nombre de la placa es `ESP32-TC-RC-BT-Simple`. El saludo es `RC:CONNECT,proto,simple`. Primero van las declaraciones de cada archivo: includes, macros, variables y tipos. Después cada función, con su código en los colores del IDE. Los dos archivos completos quedan como anexos.

Color del IDE: `void`, `if`, `#define`, `Serial.print`, `"texto"`, `BT_NAME`, `// comentario`.

Contents

1 Declaraciones de los archivos **2**

1.1 En `TelemetryChannels.h` . . . 2

1.2 En `TeleCon_Classic_Simple.ino` . . . 3

2 Qué hace cada función **6**

2.1 En `TeleCon_Classic_Simple.ino` . . . 6

2.2 En `TelemetryChannels.h` . . . 18

Anexo A `TeleCon_Classic_Simple.ino` **20**

Anexo B `TelemetryChannels.h` **28**

1 Declaraciones de los archivos

Nada de esto es una función. Es la cabecera de cada archivo: lo que el compilador ve antes del primer cuerpo. El código va con el color del IDE. Los números de línea son los del archivo.

1.1 En `TelemetryChannels.h`

El `.ino` incluye esta cabecera, así que estos nombres existen antes de `kTelemetryChannels[]`.

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  /*
6   * Phone CH1...CH8 are RC:PLOT v0...v7.
7   * Edit kTelemetryChannels[] in the sketch to choose what each channel
   reports (0-255).
8   *
9   * TEL_VEHICLE  built-in signal (temp, throttle, battery, steer, lights,
   knobs)
10  * TEL_ADC      analogRead(pin), 0-4095 scaled to 0-255
11  * TEL_GPIO     digitalRead(pin) → 0 or 255 (an output pin reports what was
   written)
12  * TEL_PWM      LEDC duty on pin. dutyMin maps to 0, dutyMax maps to 255.
13  *              8-bit motor PWM uses 0...255. 12-bit servo travel 1000-2000 µ
   s uses 204...409.
14  *              Attach that pin with ledcAttach / ledcSetup before it is read
   .
15  */
```

`#pragma once` hace que el archivo entre una sola vez en la compilación, aunque alguien lo incluya dos veces. `Arduino.h` trae `uint8_t`, `analogRead`, `digitalRead` y `ledcRead`. El comentario de en medio no genera código: recuerda que CH1 es `v0`, CH8 es `v7`, y que cada muestra es un byte 0–255.

```
17 enum TelKind : uint8_t {
18   TEL_VEHICLE = 0,
19   TEL_ADC,
20   TEL_GPIO,
21   TEL_PWM,
22 };
23
24 enum TelVehicle : uint8_t {
25   TEL_TEMP = 0,
26   TEL_THROTTLE,
27   TEL_BATT,
28   TEL_STEER,
29   TEL_LIGHTS,
30   TEL_KNOB_L,
31   TEL_KNOB_R,
32   TEL_RY,
33 };
```

`TelKind` dice de dónde sale el byte de una fila. `TEL_VEHICLE` llama a `telemetryVehicle()`.

TEL_ADC lee un pin analógico. TEL_GPIO lee un pin digital. TEL_PWM lee un duty LEDC. El valor numérico empieza en 0 y sigue en orden.

TelVehicle nombra la señal integrada, solo cuando la fila es TEL_VEHICLE. TEL_TEMP es temperatura, TEL_THROTTLE el acelerador, TEL_BATT la batería, TEL_STEER la dirección, TEL_LIGHTS las luces, TEL_KNOB_L y TEL_KNOB_R las perillas, y TEL_RY el eje Y de la palanca derecha. La tabla de fábrica no usa TEL_RY.

```
35 struct TelChannel {
36   TelKind kind;
37   TelVehicle vehicle;
38   int pin;
39   int ledcChannel;
40   int dutyMin;
41   int dutyMax;
42 };
43
44 using TelVehicleFn = uint8_t (*)(TelVehicle id);
```

TelChannel es una fila de la tabla. kind es el TelKind. vehicle es el TelVehicle; en una fila ADC, GPIO o PWM ese campo no se lee. pin es el GPIO, o −1 si la fila no tiene pin. ledcChannel es el canal LEDC del núcleo 2.x, o −1 si no hay PWM. dutyMin y dutyMax son los extremos del duty: el mínimo se manda como 0 y el máximo como 255.

TelVehicleFn es el tipo del puntero a telemetryVehicle(): recibe un TelVehicle y devuelve un byte.

1.2 En TeleCon_Classic_Simple.ino

```
1  /*
2   * TeleCon RC Vehicle Pro - Classic Simple (ESP32 Dev Module, no camera)
3   *
4   * Android (TeleCon4ESP32):
5   *   1. Pair Bluetooth "ESP32-TC-RC-BT-Simple" (PIN 1234 if asked)
6   *   2. Control Panel or RC Vehicle Pro → Board DevKit → Classic + Simple
7   *   3. Connect - phone sends RC:CONNECT,proto,simple
8   *
9   * Single-file starter for Normal users. Edit onControl() / onButton().
10  * Advanced Classic Binary: ../../Classic_Binary/
11  */
12
13 #include "BluetoothSerial.h"
14 #include "TelemetryChannels.h"
```

El bloque de comentario no entra al binario. Dice el nombre Bluetooth, el PIN y la línea de saludo. BluetoothSerial.h es el puerto Classic. TelemetryChannels.h es la cabecera de arriba.

```
16 // === CONFIG ===
17 #define BT_NAME             "ESP32-TC-RC-BT-Simple"
18 #define BT_PIN              "1234"
19 #define APP_PREFIX          "RC"
20 #define PROTO_WIRE          "simple"
21 #define SERIAL_BAUD         115200
22
```

```
// 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
#define ENABLE_DEBUG        1

#if ENABLE_DEBUG
  #define DBG_PRINT(x)      Serial.print(x)
  #define DBG_PRINTLN(x)    Serial.println(x)
  #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
#else
  #define DBG_PRINT(x)      ((void)0)
  #define DBG_PRINTLN(x)    ((void)0)
  #define DBG_PRINTF(...)   ((void)0)
#endif

#define TELEMETRY_DATA_MS   500
#define ENABLE_PLOT         1
#define PLOT_MS             50

#define ENABLE_SERIAL_INJECT 1
#define SERIAL_MAX_LINE     192

// Demo HUD values (replace with ADC reads when you wire sensors)
#define DEMO_MOTOR_TEMP     90
#define DEMO_BATT_PCT       76
```

`BT_NAME` es el nombre que Android ve al emparejar. `BT_PIN` es `1234`. `APP_PREFIX` tiene que ser `RC` y `PROTO_WIRE` tiene que ser `simple`: son las dos cadenas que acepta `handleConnect()`. `SERIAL_BAUD` es 115200 para el monitor.

`ENABLE_DEBUG` en 1 escribe en el monitor. `DBG_PRINT` pasa a `Serial.print`. `DBG_PRINTLN` pasa a `Serial.println`. `DBG_PRINTF` pasa a `Serial.printf`. En 0 esas tres macros quedan en nada y el enlace sigue igual.

`TELEMETRY_DATA_MS` es el periodo del tablero, 500 ms. `ENABLE_PLOT` en 1 envía `RC:PLOT`. `PLOT_MS` es 50 ms. `ENABLE_SERIAL_INJECT` en 1 deja pegar una línea en el monitor después del saludo. `SERIAL_MAX_LINE` corta esa línea a 192 caracteres.

`DEMO_MOTOR_TEMP` es 90 y `DEMO_BATT_PCT` es 76. Son números fijos del tablero hasta que haya un sensor. No salen de un pin.

```
// === BLUETOOTH ===
BluetoothSerial SerialBT;

bool handshakeOk = false;
bool btWasConnected = false;

int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
int g_lk = 512, g_rk = 512;
uint8_t g_sw = 0;

unsigned long lastDataMs = 0;
unsigned long lastPlotMs = 0;

#if ENABLE_SERIAL_INJECT
static String serialLineBuf;
#endif

```

```
64 static uint16_t lastPrintLX = 0xFFFF;
65 static uint16_t lastPrintLY = 0xFFFF;
66 static uint16_t lastPrintRX = 0xFFFF;
67 static uint16_t lastPrintRY = 0xFFFF;
68 static uint16_t lastPrintKnobL = 0xFFFF;
69 static uint16_t lastPrintKnobR = 0xFFFF;
70 static uint8_t lastPrintSw = 0xFF;
```

`SerialBT` es el puerto Classic. `handshakeOk` es la compuerta: en falso no hay `onControl()`, ni `RC:DATA`, ni `RC:PLOT`. `btWasConnected` guarda el cliente de la vuelta anterior, para notar una conexión o una caída una sola vez.

`g_lx`, `g_ly`, `g_rx` y `g_ry` son las palancas, de −100 a 100. Arrancan en 0. `g_ly` es el acelerador y `g_rx` la dirección. `g_lk` y `g_rk` son las perillas, de 0 a 1023, y arrancan en 512. `g_sw` es el byte de interruptores. El bit 0 son las luces.

`lastDataMs` y `lastPlotMs` son las marcas de `millis()` del tablero y del gráfico. `serialLineBuf` junta la línea que se pega en el monitor; solo existe si la inyección está compilada. Las `lastPrint*` guardan la última imagen impresa de cada eje, perilla e interruptor. Empiezan en `0xFFFF` o `0xFF` para que la primera muestra siempre se imprima. No se envían al teléfono.

```
72 String getValue(const String& line, const char* key);
73 void handleLine(const String& line);
74 void handleConnect(const String& app, const String& line);
75 void sendLine(const String& line);
76 void resetHandshake();
77 void applyFailSafe();
78 void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk,
   uint8_t sw);
79 void debugPrintButton(int id);
80 uint16_t stickTo12(int s);
81 #if ENABLE_SERIAL_INJECT
82 void serialInjectPoll();
83 void handleSerialInjectLine(const String& line);
84 #endif
```

Estas líneas no son las funciones. Son el aviso al compilador de que el cuerpo viene más abajo, para que `setup()` y `loop()` puedan llamarlas. `serialInjectPoll` y `handleSerialInjectLine` solo se declaran si `ENABLE_SERIAL_INJECT` vale 1. El cuerpo de cada una está en la sección siguiente.

```
87 // === STEERING SERVO ===
88 // DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
89 // rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
90 #ifndef ENABLE_STEER_SERVO
91 #define ENABLE_STEER_SERVO 1
92 #endif
93 #ifndef PIN_STEER_SERVO
94 #define PIN_STEER_SERVO 13
95 #endif
96 #ifndef CH_STEER_SERVO
97 #define CH_STEER_SERVO 2
98 #endif
```

`ENABLE_STEER_SERVO` en 1 arma el servo. En 0 las dos funciones del servo quedan vacías. `PIN_STEER_SERVO` es el GPIO 13. `CH_STEER_SERVO` es el canal LEDC 2, el que usa el núcleo 2.x.

El `#ifndef` deja cambiar el valor desde el IDE antes de este archivo, y si nadie lo cambió pone el de fábrica.

```
100 // Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the
     pin.
101 // Examples:
102 //   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
103 //   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
104 //   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on
     GPIO 25
105 static const TelChannel kTelemetryChannels[8] = {
106   { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
107   { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
108   { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
109   { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
110   { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
111   { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
112   { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
113   { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
114 };
```

`kTelemetryChannels` tiene ocho filas y no admite una novena. Cada llave es un `TelChannel`: tipo, señal, pin, canal LEDC, `dutyMin` y `dutyMax`. La fila 0 es CH1 (`v0`) y la fila 7 es CH8 (`v7`).

De fábrica, las filas 0 a 6 son `TEL_VEHICLE`: temperatura, acelerador, batería, dirección, luces, perilla izquierda y perilla derecha. El pin y el canal van en -1 porque la señal no sale de un GPIO. La fila 7 es `TEL_PWM` del servo: pin `PIN_STEER_SERVO`, canal `CH_STEER_SERVO`, duty de 204 a 409 (el recorrido 1000–2000 µs).

```
139 constexpr int kSteerCenterUs = 1500;
140 constexpr int kSteerTravelUs = 500;
```

Estas dos constantes están justo antes de `steerDutyFromUs()`. No están en la cabecera. `kSteerCenterUs` es 1500 µs, el centro. `kSteerTravelUs` es 500 µs a cada lado de la palanca. `rx` = 0 queda en 1500 µs.

2 Qué hace cada función

El orden sigue el de cada archivo, después de las declaraciones. Debajo de cada firma va la función entera, con el color del IDE. En el texto se usa el mismo código: el naranja es una función, del sketch o de una librería (`onControl()`, `print`, `ledcWrite`), y el morado es un nombre de este proyecto (`BT_NAME`, `g_lx`, `TEL_TEMP`).

`DBG_PRINT`, `DBG_PRINTLN` y `DBG_PRINTF` no son funciones. Con `ENABLE_DEBUG` en 1 escriben en el monitor serie. En 0 el compilador las borra.

2.1 En `TeleCon_Classic_Simple.ino`

`uint8_t telemetryVehicle(TelVehicle id)`

```
116 uint8_t telemetryVehicle(TelVehicle id) {
117   switch (id) {
118     case TEL_TEMP:
119       return (uint8_t)DEMO_MOTOR_TEMP;
```

```
120     case TEL_THROTTLE:
121       return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255)
;
122     case TEL_BATT:
123       return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0,
255);
124     case TEL_STEER:
125       return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255)
;
126     case TEL_LIGHTS:
127       return (g_sw & 0x01) ? 255 : 0;
128     case TEL_KNOB_L:
129       return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
130     case TEL_KNOB_R:
131       return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
132     case TEL_RY:
133       return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255)
;
134     default:
135       return 0;
136   }
137 }
```

Línea 116. Devuelve el byte 0–255 de una señal integrada, la que pide una fila `TEL_VEHICLE`. `TEL_TEMP` es la constante `DEMO_MOTOR_TEMP`. `TEL_THROTTLE` es la magnitud de `g_ly`. `TEL_BATT` escala `DEMO_BATT_PCT` de 0–100 a 0–255. `TEL_STEER` es la magnitud de `g_rx`. `TEL_LIGHTS` vale 255 si el bit 0 de `g_sw` está activo, y 0 si no. `TEL_KNOB_L` y `TEL_KNOB_R` llevan la perilla de 0–1023 a 0–255. `TEL_RY` es la magnitud de `g_ry`. Otro `id` devuelve 0. Acelerador, dirección y `TEL_RY` no guardan el signo: los dos extremos de la palanca dan el mismo byte.

```
142 uint32_t steerDutyFromUs(int us) {
143   int pulse = constrain(us, 500, 2500);
144   return (uint32_t)((long)pulse * 4095L /
 20000L);
145 }
```

uint32_t steerDutyFromUs(int us)

Línea 142. Pasa un ancho de pulso, en microsegundos, a duty LEDC de 12 bits: pulso $\times 4095/20000$. Antes limita el pulso a 500–2500 µs. 1500 µs queda en el centro de un marco de 20 ms.

```
147 uint32_t steerDutyFromRx(int rx) {
148   int s = constrain(rx, -100, 100);
149   int us = (int)map(s, -100, 100,
kSteerCenterUs - kSteerTravelUs,
kSteerCenterUs + kSteerTravelUs);
150   return steerDutyFromUs(us);
151 }
```

uint32_t steerDutyFromRx(int rx)

Línea 147. Limita `rx` a $-100 \ldots 100$, lo convierte en microsegundos (0 es 1500 µs, a fondo ±500 µs) y llama a `steerDutyFromUs`. El trim del teléfono ya viene dentro de `rx`.

void steerServoBegin()

```
153 #if ENABLE_STEER_SERVO
```

```
154 void steerServoBegin() {
155 #if ESP_ARDUINO_VERSION_MAJOR >= 3
156   ledcAttach(PIN_STEER_SERVO, 50, 12);
157   ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
158 #else
159   ledcSetup(CH_STEER_SERVO, 50, 12);
160   ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
161   ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
162 #endif
163 }
```

```
174 void steerServoBegin() {}
```

Línea 154, o la línea 174 si `ENABLE_STEER_SERVO` es 0. Con el servo activo, deja un PWM de 50 Hz y 12 bits en `PIN_STEER_SERVO` y escribe el centro. En el núcleo 3.x el canal se une por número de pin (`ledcAttach`, `ledcWrite`). En el 2.x se usa `CH_STEER_SERVO` (`ledcSetup`, `ledcAttachPin`, `ledcWrite`). Con el interruptor en 0 la función no hace nada.

void steerServoWriteRx(int rx)

```
165 void steerServoWriteRx(int rx) {
166   uint32_t duty = steerDutyFromRx(rx);
167 #if ESP_ARDUINO_VERSION_MAJOR >= 3
168   ledcWrite(PIN_STEER_SERVO, duty);
169 #else
170   ledcWrite(CH_STEER_SERVO, duty);
171 #endif
172 }
```

```
175 void steerServoWriteRx(int rx) { (void)rx; }
```

Línea 165, o la línea 175 si el servo está apagado. Calcula el duty de `rx` y lo escribe. La versión vacía solo descarta `rx`, para que el compilador no avise de un parámetro sin uso. `applyFailSafe()` la llama con 0, que es el centro.

void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw)

```
178 void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t
    sw) {
179   debugPrintControl(lx, ly, rx, ry, lk, rk, sw);
180
181   // rx = 0 → 1500 µs. Trim ticks on the phone already shift this
    value.
182   steerServoWriteRx(rx);
183
184   // YOUR CODE HERE - drive motors / other actuators from sticks &
    knobs
185   //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 =
    lights
186   (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
187 }
```

Línea 178. Corre una vez por cada línea `RC:CTRL`. Imprime el cambio en el monitor, mueve el servo con `rx`, y deja el bloque `YOUR CODE HERE` para lo que deba seguir la palanca: PWM de motores, un servo de paneo desde `rk`, un pin de luces desde `sw & 0x01`. Los casteos (`void`) callan el aviso de los valores que el arranque todavía no usa. Se borra el casteo del valor que se empiece a usar.

void onButton(int id)

```
189 void onButton(int id) {
190   debugPrintButton(id);
191
192   // YOUR CODE HERE - button actions
193   if (id == 2) {
194     // Horn note - pulse buzzer briefly
195   } else if (id == 4) {
196     // Center pan note - move camera pan servo to center
197   }
198 }
```

Línea 189. Corre una vez por cada `RC:BTN`. Imprime el id. El comentario de fábrica deja el id 2 para un pulso corto de bocina y el id 4 para centrar un servo de paneo. Esos dos cuerpos están vacíos. Un botón que tenga que quedarse abajo todo el tiempo encaja mejor como bit de `sw` dentro de `onControl()`, porque `RC:BTN` es la pulsación, no el nivel sostenido.

void setup()

```
200 void setup() {
201   Serial.begin(SERIAL_BAUD);
202   delay(200);
203
204   steerServoBegin();
205
206   // Legacy PIN pairing only (Android often fails if SSP + setPin are
    mixed).
207   SerialBT.disableSSP();
208   SerialBT.setPin(BT_PIN, 4);
209
210   // isMaster=false, disableBLE=true → Classic SPP only
211   if (!SerialBT.begin(BT_NAME, false, true)) {
212     Serial.println("Bluetooth init failed!");
213     while (true) { delay(1000); }
214   }
215
216   DBG_PRINTLN();
217   DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
218   DBG_PRINT("BT name: ");
219   DBG_PRINTLN(BT_NAME);
220   DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
221   DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal →
    Classic + Simple");
222   DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
223   DBG_PRINTLN("Examples:");
```

```
224   DBG_PRINTLN("  RC:CONNECT,proto,simple");
225   DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
226   DBG_PRINTLN("  RC:BTN,id,1");
227   DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
228   DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
229   DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|,
    demo batt/temp).");
230 #if ENABLE_PLOT
231   DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit
    kTelemetryChannels[]).");
232 #else
233   DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
234 #endif
235 #if ENABLE_SERIAL_INJECT
236   DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending =
    Newline):");
237   DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog
    ,90,batt,76,led,01");
238   DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0
    ");
239   DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes
    after.");
240 #endif
241   DBG_PRINTLN();
242 }
```

Línea 200. Arranca el serie a 115200, espera 200 ms, centra el servo, apaga Secure Simple Pairing, fija el PIN `1234` y abre Bluetooth Classic con `BT_NAME`, como esclavo y sin BLE. Si `SerialBT.begin` falla, imprime el fallo y se queda en `while (true)`. El cartel de después es solo depuración: nombre, saludo y líneas de ejemplo para el monitor.

void loop()

```
244 void loop() {
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple
    ");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
257
258   while (SerialBT.available()) {
259     String line = SerialBT.readStringUntil('\n');
260     line.trim();
261     if (line.length() > 0) {
```

```
262       handleLine(line);
263     }
264   }
265
266 #if ENABLE_SERIAL_INJECT
267   serialInjectPoll();
268 #endif
269
270   if (!handshakeOk) return;
271
272   unsigned long now = millis();
273
274   if (now - lastDataMs >= TELEMETRY_DATA_MS) {
275     lastDataMs = now;
276
277     int speedX10 = abs(g_ly) * 25 / 10;
278     int analog = DEMO_MOTOR_TEMP;
279     int batt = DEMO_BATT_PCT;
280
281     char data[128];
282     snprintf(data, sizeof(data),
283       "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,
    led,%02X",
284       speedX10, analog, batt, g_sw);
285     sendLine(data);
286   }
287
288 #if ENABLE_PLOT
289   if (now - lastPlotMs >= PLOT_MS) {
290     lastPlotMs = now;
291
292     uint8_t sample[8];
293     telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
294     char plot[128];
295     snprintf(plot, sizeof(plot),
296       "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
297       sample[0], sample[1], sample[2], sample[3],
298       sample[4], sample[5], sample[6], sample[7]);
299     sendLine(plot);
300   }
301 #endif
302 }
```

Línea 244. Arduino llama a `loop()` una vez tras otra, desde que termina `setup()`. Cada vuelta sigue estos pasos, y el siguiente empieza cuando el anterior ya terminó.

```
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected -
waiting for RC:CONNECT,proto,simple");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
```

1. Cliente Bluetooth

Lee `SerialBT.hasClient()` y lo compara con `btWasConnected`. Si cambió, guarda el estado nuevo. Si el teléfono acaba de entrar, llama a `resetHandshake()`: `handshakeOk` queda en falso, y el monitor avisa que espera `RC:CONNECT,proto,simple`. Si acaba de salir, llama a `applyFailSafe()` (palancas en 0, perillas en 512, interruptores en 0, servo al centro) y después a `resetHandshake()`. Si el cliente sigue igual, este bloque no llama a nadie.

```
258   while (SerialBT.available()) {
259     String line = SerialBT.
readStringUntil('\n');
260     line.trim();
261     if (line.length() > 0) {
262       handleLine(line);
263     }
264   }
```

2. Líneas de Bluetooth

Mientras `SerialBT` tenga bytes, lee hasta `\n`, quita espacios con `trim()` y, si queda texto, llama a `handleLine()`. Una línea `RC:CONNECT,proto,simple` válida en esta misma vuelta pone `handshakeOk` en verdadero antes de los temporizadores de abajo.

```
266 #if ENABLE_SERIAL_INJECT
267   serialInjectPoll();
268 #endif
```

3. Monitor serie

Con `ENABLE_SERIAL_INJECT` en 1, llama a `serialInjectPoll()`. Una línea del monitor que empiece por `RC:DATA` o por `RC:PLOT` sale al teléfono en ese momento. Con el interruptor en 0, este bloque no entra al binario.

```
270   if (!handshakeOk) return;
```

4. Compuerta del saludo

Si `handshakeOk` es falso, `loop()` termina en ese `return`. El tablero y el gráfico esperan la respuesta de `handleConnect()`: `RC:ACK,app,RC`.

```
272   unsigned long now = millis();
```

5. Reloj

Guarda `millis()` en `now`. El tablero y el gráfico usan este mismo instante, cada uno con su marca. Cumplir un plazo no reinicia el otro. Los dos pueden cumplirse en la misma vuelta: primero el tablero, después el gráfico.

```
274   if (now - lastDataMs >=
TELEMETRY_DATA_MS) {
275     lastDataMs = now;
276
277     int speedX10 = abs(g_ly) * 25 / 10;
278     int analog = DEMO_MOTOR_TEMP;
279     int batt = DEMO_BATT_PCT;
280
281     char data[128];
282     snprintf(data, sizeof(data),
283       "RC:DATA,left,%d,right,0,lo,1,ro,0,
lg,1,rg,0,analog,%d,batt,%d,led,%02X",
284       speedX10, analog, batt, g_sw);
285     sendLine(data);
286   }
```

6. Tablero, cada TELEMETRY_DATA_MS (500 ms)

Cuando now − lastDataMs llega al plazo, guarda now en lastDataMs y arma una línea RC:DATA. left es abs(g_ly) * 25 / 10: acelerador 100 sale como 250, y el teléfono lo lee como velocidad ×10. abs() manda la magnitud, así que acelerador a fondo hacia atrás también sale como 250. right va fijo en 0. lo,1,ro,0,lg,1,rg,0 son las lámparas del panel, también fijas. analog copia DEMO_MOTOR_TEMP (90) y batt copia DEMO_BATT_PCT (76). led es g_sw en dos dígitos hexadecimales. sendLine() manda esa línea.

```
288 #if ENABLE_PLOT
289   if (now - lastPlotMs >= PLOT_MS) {
290     lastPlotMs = now;
291
292     uint8_t sample[8];
293     telemetryReadAll(sample,
kTelemetryChannels, telemetryVehicle);
294     char plot[128];
295     snprintf(plot, sizeof(plot),
296       "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4
,%u,v5,%u,v6,%u,v7,%u",
297       sample[0], sample[1], sample[2],
sample[3],
298       sample[4], sample[5], sample[6],
sample[7]);
299     sendLine(plot);
300   }
301 #endif
```

7. Gráfico, cada PLOT_MS (50 ms)

Este bloque entra al binario cuando ENABLE_PLOT vale 1. Cuando now − lastPlotMs llega al plazo, guarda now en lastPlotMs, llama a telemetryReadAll() con kTelemetryChannels y telemetryVehicle, y manda RC:PLOT de v0 a v7, en el orden de las ocho filas. Con ENABLE_PLOT en 0, este bloque no entra al binario.

void handleSerialInjectLine(const String& line)

```
304 #if ENABLE_SERIAL_INJECT
305 void handleSerialInjectLine(const String& line) {
306   if (!SerialBT.hasClient()) {
307     DBG_PRINTLN("[SERIAL] no BT client - line not sent");
308     return;
309   }
310   if (!handshakeOk) {
311     DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before
inject");
312     return;
313   }
314
315   if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
316     sendLine(line);
```

```
317     DBG_PRINT("[SERIAL] inject one-shot → phone: ");
318     DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
319     return;
320   }
321
322   DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
323   DBG_PRINTLN(line);
324 }
```

Línea 305. El sketch de fábrica la compila, porque `ENABLE_SERIAL_INJECT` vale 1. Si no hay cliente Bluetooth, o el saludo todavía no está listo, no envía la línea. Si empieza por `RC:DATA` o por `RC:PLOT`, la manda una sola vez con `sendLine`. Otra línea solo se anota en el monitor. Los temporizadores de `loop()` siguen, así el paquete automático siguiente reemplaza esa línea en el teléfono.

void serialInjectPoll()

```
326 void serialInjectPoll() {
327   while (Serial.available()) {
328     char c = (char)Serial.read();
329     if (c == '\n') {
330       if (serialLineBuf.length() > 0) {
331         serialLineBuf.trim();
332         if (serialLineBuf.length() > 0) {
333           handleSerialInjectLine(serialLineBuf);
334         }
335         serialLineBuf = "";
336       }
337     } else if (c != '\r') {
338       if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
339         serialLineBuf += c;
340       } else {
341         serialLineBuf = "";
342         DBG_PRINTLN("[SERIAL] line too long, discarded");
343       }
344     }
345   }
346 }
347 #endif
```

Línea 326. Lee el monitor carácter a carácter hasta `\n`. Ignora `\r`, para que sirva un fin de línea “Both NL & CR”. Al cerrar la línea la recorta y, si queda texto, llama a `handleSerialInjectLine`. Si el buffer pasa de `SERIAL_MAX_LINE` (192), lo tira y avisa.

```
349 uint16_t stickTo12(int s) {
350   return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
351 }
```

uint16_t stickTo12(int s)

Línea 349. Lleva una palanca de $-100 \ldots 100$ a $0 \ldots 4095$, solo para el texto del monitor. 0 en la palanca queda cerca de 2048. Ese número no es el duty del servo ni el byte de `RC:PLOT`.

void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw)

```
353 void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk,
    uint8_t sw) {
354   uint16_t lx12 = stickTo12(lx);
355   uint16_t ly12 = stickTo12(ly);
356   uint16_t rx12 = stickTo12(rx);
357   uint16_t ry12 = stickTo12(ry);
358   uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
359   uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);
360
361   if (lx12 != lastPrintLX) {
362     DBG_PRINTF("L Stick X: %u    ", lx12);
363     DBG_PRINTF("L Stick Y: %u\n", ly12);
364     lastPrintLX = lx12;
365   }
366   if (ly12 != lastPrintLY) {
367     DBG_PRINTF("L Stick X: %u    ", lx12);
368     DBG_PRINTF("L Stick Y: %u\n", ly12);
369     lastPrintLY = ly12;
370   }
371   if (rx12 != lastPrintRX) {
372     DBG_PRINTF("R Stick X: %u    ", rx12);
373     DBG_PRINTF("R Stick Y: %u\n", ry12);
374     lastPrintRX = rx12;
375   }
376   if (ry12 != lastPrintRY) {
377     DBG_PRINTF("R Stick X: %u    ", rx12);
378     DBG_PRINTF("R Stick Y: %u\n", ry12);
379     lastPrintRY = ry12;
380   }
381   if (lkU != lastPrintKnobL) {
382     DBG_PRINTF("Left Knob: %u\n", lkU);
383     lastPrintKnobL = lkU;
384   }
385   if (rkU != lastPrintKnobR) {
386     DBG_PRINTF("Right Knob: %u\n", rkU);
387     lastPrintKnobR = rkU;
388   }
389   if (sw != lastPrintSw) {
390     for (int i = 0; i < 6; i++) {
391       byte bit = (1 << i);
392       if ((sw & bit) != (lastPrintSw & bit)) {
393         DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
394       }
395     }
396     lastPrintSw = sw;
397   }
398 }
```

Línea 353. Pasa cada palanca por `stickTo12` y cada perilla por un límite 0–1023. Imprime un eje solo cuando ese valor cambió respecto de `lastPrint*`, así una palanca quieta no

llena el monitor. Si cambia uno de los seis primeros bits de `sw`, imprime `SW1` a `SW6`. Con `ENABLE_DEBUG` en 0 la función sigue corriendo y las macros no escriben nada.

```
400 void debugPrintButton(int id) {
401   DBG_PRINTF("BTN%d : Pressed\n", id);
402 }
```

void debugPrintButton(int id)

Línea 400. Imprime `BTNn : Pressed` con el id que llegó en `RC:BTN`.

```
404 void sendLine(const String& line) {
405   SerialBT.print(line);
406   SerialBT.print('\n');
407 }
```

void sendLine(const String& line)

Línea 404. Único envío hacia el teléfono. Escribe el texto y después un salto de línea, por `SerialBT`. El saludo, el NAK, el tablero y el gráfico pasan todos por aquí.

```
409 void applyFailSafe() {
410   g_lx = g_ly = g_rx = g_ry = 0;
411   g_lk = g_rk = 512;
412   g_sw = 0;
413   steerServoWriteRx(0);
414   DBG_PRINTLN("[IO] fail-safe");
415   // YOUR CODE HERE - stop motors on
    disconnect
416 }
```

void applyFailSafe()

Línea 409. Deja las cuatro palancas en 0, las dos perillas en 512 y `g_sw` en 0, y escribe el servo con `rx` 0 (centro). Debajo del segundo `YOUR CODE HERE` se paran los motores que `onControl()` haya puesto en marcha. Sin eso, el último acelerador sigue aplicado cuando el teléfono ya no está.

```
418 void resetHandshake() {
419   handshakeOk = false;
420   lastPrintLX = lastPrintLY = lastPrintRX
     = lastPrintRY = 0xFFFF;
421   lastPrintKnobL = lastPrintKnobR = 0
    xFFFF;
422   lastPrintSw = 0xFF;
423 }
```

void resetHandshake()

Línea 418. Pone `handshakeOk` en falso y olvida los últimos valores del monitor, para que la sesión nueva vuelva a imprimirlos. No mueve el servo ni las salidas: quien vio la caída ya llamó a `applyFailSafe()`.

String getValue(const String& line, const char* key)

```
425 String getValue(const String& line, const char* key) {
426   int colon = line.indexOf(':');
427   if (colon < 0) return "";
428
429   int pos = line.indexOf(',', colon);
430   if (pos < 0) return "";
431   pos++;
432
433   while (pos < (int)line.length()) {
434     int keyEnd = line.indexOf(',', pos);
```

```
435     if (keyEnd < 0) break;
436
437     String k = line.substring(pos, keyEnd);
438     int valEnd = line.indexOf(',', keyEnd + 1);
439     String v = (valEnd < 0)
440       ? line.substring(keyEnd + 1)
441       : line.substring(keyEnd + 1, valEnd);
442
443     if (k == key) return v;
444     pos = (valEnd < 0) ? line.length() : valEnd + 1;
445   }
446   return "";
447 }
```

Línea 425. Saca el valor de una clave en una línea `APP:TIPO,clave,valor,clave,valor`. Salta la cabeza hasta la primera coma y recorre los pares. El valor llega hasta la coma siguiente, o hasta el fin de la línea en el último par. Las claves distinguen mayúsculas (`lx`, no `LX`). Si la clave no está, devuelve una cadena vacía, y `toInt()` de esa cadena es 0.

`void handleConnect(const String& app, const String& line)`

```
449 void handleConnect(const String& app, const String& line) {
450   if (!app.equalsIgnoreCase(APP_PREFIX)) {
451     sendLine(String("RC:NAK,reason,app_mismatch,expected,") +
APP_PREFIX +
452              ",actual," + app);
453     handshakeOk = false;
454     return;
455   }
456
457   String proto = getValue(line, "proto");
458   if (proto.length() == 0) {
459     sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
PROTO_WIRE +
460              ",actual,unknown");
461     handshakeOk = false;
462     return;
463   }
464
465   if (proto.equalsIgnoreCase(PROTO_WIRE)) {
466     sendLine("RC:ACK,app,RC");
467     handshakeOk = true;
468     DBG_PRINTLN("[HS] ACK - session ready");
469     return;
470   }
471
472   sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
PROTO_WIRE +
473            ",actual," + proto);
474   handshakeOk = false;
475 }
```

Línea 449. Exige dos cosas, sin distinguir mayúsculas. El nombre antes de los dos puntos

tiene que ser APP_PREFIX (RC). El valor proto tiene que ser PROTO_WIRE (simple). Si el nombre no coincide, responde RC:NAK con app_mismatch y deja handshakeOk en falso. Si falta proto, o trae otro valor, responde proto_mismatch. Si ambos coinciden, envía exactamente RC:ACK,app,RC y pone handshakeOk en verdadero.

void handleLine(const String& line)

```
477 void handleLine(const String& line) {
478   int colon = line.indexOf(':');
479   if (colon <= 0) return;
480
481   String app = line.substring(0, colon);
482   String rest = line.substring(colon + 1);
483   int comma = rest.indexOf(',');
484   String type = (comma < 0) ? rest : rest.substring(0, comma);
485
486   if (type.equalsIgnoreCase("CONNECT")) {
487     handleConnect(app, line);
488     return;
489   }
490
491   if (!handshakeOk) return;
492   if (!app.equalsIgnoreCase("RC")) return;
493
494   if (type.equalsIgnoreCase("CTRL")) {
495     g_lx = getValue(line, "lx").toInt();
496     g_ly = getValue(line, "ly").toInt();
497     g_rx = getValue(line, "rx").toInt();
498     g_ry = getValue(line, "ry").toInt();
499     g_lk = getValue(line, "lk").toInt();
500     g_rk = getValue(line, "rk").toInt();
501     String swHex = getValue(line, "sw");
502     g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
503
504     onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505   } else if (type.equalsIgnoreCase("BTN")) {
506     onButton(getValue(line, "id").toInt());
507   }
508 }
```

Línea 477. Si la línea no tiene dos puntos, vuelve. Separa la app y el tipo (el texto hasta la primera coma). CONNECT llama a handleConnect aunque el saludo no esté listo. Cualquier otro tipo se ignora mientras handshakeOk es falso, y también si la app no es RC. CTRL guarda lx, ly, rx, ry, lk, rk y sw (el campo sw se lee en hexadecimal) y llama a onControl. BTN llama a onButton con el id. Una línea DATA, PLOT o SET que llegue del teléfono no hace nada: este sketch las envía, no las obedece.

2.2 En TelemetryChannels.h

El archivo declara los tipos que usan las tres funciones. TelKind es el origen de la fila: vehículo, ADC, GPIO o PWM. TelVehicle es la señal integrada (TEL_TEMP, TEL_THROTTLE, TEL_BATT, TEL_STEER, TEL_LIGHTS, TEL_KNOB_L, TEL_KNOB_R, TEL_RY). TelChannel guarda el tipo, la señal,

el pin, el canal LEDC y el rango de duty. `TelVehicleFn` es el puntero a `telemetryVehicle()`.

`uint8_t telemetryReadPwm(const TelChannel& ch)`

```
46 inline uint8_t telemetryReadPwm(const TelChannel& ch) {
47   if (ch.dutyMax <= ch.dutyMin) return 0;
48   uint32_t duty = 0;
49 #if ESP_ARDUINO_VERSION_MAJOR >= 3
50   if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
51 #else
52   if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
53 #endif
54   long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
55   return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
56 }
```

Línea 46. Si `dutyMax` no es mayor que `dutyMin`, devuelve 0. Lee el duty que ya está corriendo: en el núcleo 3.x con `ledcRead` del número de pin, en el 2.x con `ledcRead` del número de canal. Recorta ese duty a `dutyMin`…`dutyMax` y lo escala a 0–255. La fila de fábrica del servo usa 204…409, que es el recorrido 1000–2000 µs; el centro sale cerca de 128. El pin tiene que estar unido en `setup` antes de que esta función lo lea.

`uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle)`

```
58 inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn
vehicle) {
59   switch (ch.kind) {
60     case TEL_ADC:
61       if (ch.pin < 0) return 0;
62       return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0,
4095, 0, 255);
63     case TEL_GPIO:
64       if (ch.pin < 0) return 0;
65       return digitalRead(ch.pin) ? 255 : 0;
66     case TEL_PWM:
67       return telemetryReadPwm(ch);
68     default:
69       return vehicle ? vehicle(ch.vehicle) : 0;
70   }
71 }
```

Línea 58. Un byte, según `ch.kind`. `TEL_ADC` lee el pin y escala 0–4095 a 0–255; un pin negativo devuelve 0. `TEL_GPIO` devuelve 255 o 0 según `digitalRead`; un pin de salida informa el nivel que se escribió. `TEL_PWM` llama a `telemetryReadPwm`. Cualquier otro tipo llama a `vehicle(ch.vehicle)`. Si ese puntero es nulo, devuelve 0.

```
73 inline void telemetryReadAll(uint8_t out
   [8], const TelChannel table[8],
   TelVehicleFn vehicle) {
74   for (int i = 0; i < 8; i++) out[i] =
   telemetryReadOne(table[i], vehicle);
75 }
```

void telemetryReadAll()

Línea 73. Recorre las ocho filas de `kTelemetryChannels[]` y escribe `out[0]` hasta `out[7]`. `loop()` manda esos bytes como `v0` hasta `v7`. La fila 0 es CH1 del teléfono y la fila 7 es CH8. No hay noveno puesto en esta función.

Anexo A TeleCon_Classic_Simple.ino

Este es el sketch entero, el archivo que abre el IDE. Los números de línea son los del archivo.

```
1  /*
2   * TeleCon RC Vehicle Pro - Classic Simple (ESP32 Dev Module, no camera)
3   *
4   * Android (TeleCon4ESP32):
5   *   1. Pair Bluetooth "ESP32-TC-RC-BT-Simple" (PIN 1234 if asked)
6   *   2. Control Panel or RC Vehicle Pro → Board DevKit → Classic + Simple
7   *   3. Connect - phone sends RC:CONNECT,proto,simple
8   *
9   * Single-file starter for Normal users. Edit onControl() / onButton().
10  * Advanced Classic Binary: ../../Classic_Binary/
11  */
12
13 #include "BluetoothSerial.h"
14 #include "TelemetryChannels.h"
15
16 // === CONFIG ===
17 #define BT_NAME             "ESP32-TC-RC-BT-Simple"
18 #define BT_PIN              "1234"
19 #define APP_PREFIX          "RC"
20 #define PROTO_WIRE          "simple"
21 #define SERIAL_BAUD         115200
22
23 // 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
24 #define ENABLE_DEBUG        1
25
26 #if ENABLE_DEBUG
27   #define DBG_PRINT(x)      Serial.print(x)
28   #define DBG_PRINTLN(x)    Serial.println(x)
29   #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
30 #else
31   #define DBG_PRINT(x)      ((void)0)
32   #define DBG_PRINTLN(x)    ((void)0)
33   #define DBG_PRINTF(...)   ((void)0)
34 #endif
35
36 #define TELEMETRY_DATA_MS   500
37 #define ENABLE_PLOT         1
38 #define PLOT_MS             50
39
40 #define ENABLE_SERIAL_INJECT 1
41 #define SERIAL_MAX_LINE     192
42
43 // Demo HUD values (replace with ADC reads when you wire sensors)
44 #define DEMO_MOTOR_TEMP     90
45 #define DEMO_BATT_PCT       76
46
47 // === BLUETOOTH ===
48 BluetoothSerial SerialBT;
```

```
49
50  bool handshakeOk = false;
51  bool btWasConnected = false;
52
53  int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
54  int g_lk = 512, g_rk = 512;
55  uint8_t g_sw = 0;
56
57  unsigned long lastDataMs = 0;
58  unsigned long lastPlotMs = 0;
59
60  #if ENABLE_SERIAL_INJECT
61  static String serialLineBuf;
62  #endif
63
64  static uint16_t lastPrintLX = 0xFFFF;
65  static uint16_t lastPrintLY = 0xFFFF;
66  static uint16_t lastPrintRX = 0xFFFF;
67  static uint16_t lastPrintRY = 0xFFFF;
68  static uint16_t lastPrintKnobL = 0xFFFF;
69  static uint16_t lastPrintKnobR = 0xFFFF;
70  static uint8_t lastPrintSw = 0xFF;
71
72  String getValue(const String& line, const char* key);
73  void handleLine(const String& line);
74  void handleConnect(const String& app, const String& line);
75  void sendLine(const String& line);
76  void resetHandshake();
77  void applyFailSafe();
78  void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw);
79  void debugPrintButton(int id);
80  uint16_t stickTo12(int s);
81  #if ENABLE_SERIAL_INJECT
82  void serialInjectPoll();
83  void handleSerialInjectLine(const String& line);
84  #endif
85
86
87  // === STEERING SERVO ===
88  // DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
89  // rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
90  #ifndef ENABLE_STEER_SERVO
91  #define ENABLE_STEER_SERVO 1
92  #endif
93  #ifndef PIN_STEER_SERVO
94  #define PIN_STEER_SERVO 13
95  #endif
96  #ifndef CH_STEER_SERVO
97  #define CH_STEER_SERVO 2
98  #endif
99
100 // Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the pin.
101 // Examples:
102 //   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
103 //   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
104 //   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on GPIO 25
105 static const TelChannel kTelemetryChannels[8] = {
106   { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
107   { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
108   { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
109   { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
110   { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
111   { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
112   { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
113   { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
```

```
};

uint8_t telemetryVehicle(TelVehicle id) {
  switch (id) {
    case TEL_TEMP:
      return (uint8_t)DEMO_MOTOR_TEMP;
    case TEL_THROTTLE:
      return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255);
    case TEL_BATT:
      return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0, 255);
    case TEL_STEER:
      return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255);
    case TEL_LIGHTS:
      return (g_sw & 0x01) ? 255 : 0;
    case TEL_KNOB_L:
      return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
    case TEL_KNOB_R:
      return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
    case TEL_RY:
      return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255);
    default:
      return 0;
  }
}

constexpr int kSteerCenterUs = 1500;
constexpr int kSteerTravelUs = 500;

uint32_t steerDutyFromUs(int us) {
  int pulse = constrain(us, 500, 2500);
  return (uint32_t)((long)pulse * 4095L / 20000L);
}

uint32_t steerDutyFromRx(int rx) {
  int s = constrain(rx, -100, 100);
  int us = (int)map(s, -100, 100, kSteerCenterUs - kSteerTravelUs, kSteerCenterUs +
kSteerTravelUs);
  return steerDutyFromUs(us);
}

#if ENABLE_STEER_SERVO
void steerServoBegin() {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(PIN_STEER_SERVO, 50, 12);
  ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#else
  ledcSetup(CH_STEER_SERVO, 50, 12);
  ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
  ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#endif
}

void steerServoWriteRx(int rx) {
  uint32_t duty = steerDutyFromRx(rx);
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(PIN_STEER_SERVO, duty);
#else
  ledcWrite(CH_STEER_SERVO, duty);
#endif
}
#else
void steerServoBegin() {}
void steerServoWriteRx(int rx) { (void)rx; }
#endif

```

```
void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
  debugPrintControl(lx, ly, rx, ry, lk, rk, sw);

  // rx = 0 → 1500 µs. Trim ticks on the phone already shift this value.
  steerServoWriteRx(rx);

  // YOUR CODE HERE - drive motors / other actuators from sticks & knobs
  //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 = lights
  (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
}

void onButton(int id) {
  debugPrintButton(id);

  // YOUR CODE HERE - button actions
  if (id == 2) {
    // Horn note - pulse buzzer briefly
  } else if (id == 4) {
    // Center pan note - move camera pan servo to center
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  delay(200);

  steerServoBegin();

  // Legacy PIN pairing only (Android often fails if SSP + setPin are mixed).
  SerialBT.disableSSP();
  SerialBT.setPin(BT_PIN, 4);

  // isMaster=false, disableBLE=true → Classic SPP only
  if (!SerialBT.begin(BT_NAME, false, true)) {
    Serial.println("Bluetooth init failed!");
    while (true) { delay(1000); }
  }

  DBG_PRINTLN();
  DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
  DBG_PRINT("BT name: ");
  DBG_PRINTLN(BT_NAME);
  DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
  DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal → Classic + Simple");
  DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
  DBG_PRINTLN("Examples:");
  DBG_PRINTLN("  RC:CONNECT,proto,simple");
  DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
  DBG_PRINTLN("  RC:BTN,id,1");
  DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
  DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
  DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|, demo batt/temp).");
#if ENABLE_PLOT
  DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit kTelemetryChannels[]).");
#else
  DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
#endif
#if ENABLE_SERIAL_INJECT
  DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending = Newline):");
  DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog,90,batt,76,led,01");
  DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0");
  DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes after.");
#endif
  DBG_PRINTLN();
}
```

```
243
244 void loop() {
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
257
258   while (SerialBT.available()) {
259     String line = SerialBT.readStringUntil('\n');
260     line.trim();
261     if (line.length() > 0) {
262       handleLine(line);
263     }
264   }
265
266 #if ENABLE_SERIAL_INJECT
267   serialInjectPoll();
268 #endif
269
270   if (!handshakeOk) return;
271
272   unsigned long now = millis();
273
274   if (now - lastDataMs >= TELEMETRY_DATA_MS) {
275     lastDataMs = now;
276
277     int speedX10 = abs(g_ly) * 25 / 10;
278     int analog = DEMO_MOTOR_TEMP;
279     int batt = DEMO_BATT_PCT;
280
281     char data[128];
282     snprintf(data, sizeof(data),
283       "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,led,%02X",
284       speedX10, analog, batt, g_sw);
285     sendLine(data);
286   }
287
288 #if ENABLE_PLOT
289   if (now - lastPlotMs >= PLOT_MS) {
290     lastPlotMs = now;
291
292     uint8_t sample[8];
293     telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
294     char plot[128];
295     snprintf(plot, sizeof(plot),
296       "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
297       sample[0], sample[1], sample[2], sample[3],
298       sample[4], sample[5], sample[6], sample[7]);
299     sendLine(plot);
300   }
301 #endif
302 }
303
304 #if ENABLE_SERIAL_INJECT
305 void handleSerialInjectLine(const String& line) {
306   if (!SerialBT.hasClient()) {
307     DBG_PRINTLN("[SERIAL] no BT client - line not sent");
```

```
    return;
  }
  if (!handshakeOk) {
    DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before inject");
    return;
  }

  if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
    sendLine(line);
    DBG_PRINT("[SERIAL] inject one-shot → phone: ");
    DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
    return;
  }

  DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
  DBG_PRINTLN(line);
}

void serialInjectPoll() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\n') {
      if (serialLineBuf.length() > 0) {
        serialLineBuf.trim();
        if (serialLineBuf.length() > 0) {
          handleSerialInjectLine(serialLineBuf);
        }
        serialLineBuf = "";
      }
    } else if (c != '\r') {
      if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
        serialLineBuf += c;
      } else {
        serialLineBuf = "";
        DBG_PRINTLN("[SERIAL] line too long, discarded");
      }
    }
  }
}
#endif

uint16_t stickTo12(int s) {
  return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
}

void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
  uint16_t lx12 = stickTo12(lx);
  uint16_t ly12 = stickTo12(ly);
  uint16_t rx12 = stickTo12(rx);
  uint16_t ry12 = stickTo12(ry);
  uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
  uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);

  if (lx12 != lastPrintLX) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLX = lx12;
  }
  if (ly12 != lastPrintLY) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLY = ly12;
  }
  if (rx12 != lastPrintRX) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
```

```
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRX = rx12;
  }
  if (ry12 != lastPrintRY) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRY = ry12;
  }
  if (lkU != lastPrintKnobL) {
    DBG_PRINTF("Left Knob: %u\n", lkU);
    lastPrintKnobL = lkU;
  }
  if (rkU != lastPrintKnobR) {
    DBG_PRINTF("Right Knob: %u\n", rkU);
    lastPrintKnobR = rkU;
  }
  if (sw != lastPrintSw) {
    for (int i = 0; i < 6; i++) {
      byte bit = (1 << i);
      if ((sw & bit) != (lastPrintSw & bit)) {
        DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
      }
    }
    lastPrintSw = sw;
  }
}

void debugPrintButton(int id) {
  DBG_PRINTF("BTN%d : Pressed\n", id);
}

void sendLine(const String& line) {
  SerialBT.print(line);
  SerialBT.print('\n');
}

void applyFailSafe() {
  g_lx = g_ly = g_rx = g_ry = 0;
  g_lk = g_rk = 512;
  g_sw = 0;
  steerServoWriteRx(0);
  DBG_PRINTLN("[IO] fail-safe");
  // YOUR CODE HERE - stop motors on disconnect
}

void resetHandshake() {
  handshakeOk = false;
  lastPrintLX = lastPrintLY = lastPrintRX = lastPrintRY = 0xFFFF;
  lastPrintKnobL = lastPrintKnobR = 0xFFFF;
  lastPrintSw = 0xFF;
}

String getValue(const String& line, const char* key) {
  int colon = line.indexOf(':');
  if (colon < 0) return "";

  int pos = line.indexOf(',', colon);
  if (pos < 0) return "";
  pos++;

  while (pos < (int)line.length()) {
    int keyEnd = line.indexOf(',', pos);
    if (keyEnd < 0) break;

    String k = line.substring(pos, keyEnd);
```

```
    int valEnd = line.indexOf(',', keyEnd + 1);
    String v = (valEnd < 0)
      ? line.substring(keyEnd + 1)
      : line.substring(keyEnd + 1, valEnd);

    if (k == key) return v;
    pos = (valEnd < 0) ? line.length() : valEnd + 1;
  }
  return "";
}

void handleConnect(const String& app, const String& line) {
  if (!app.equalsIgnoreCase(APP_PREFIX)) {
    sendLine(String("RC:NAK,reason,app_mismatch,expected,") + APP_PREFIX +
             ",actual," + app);
    handshakeOk = false;
    return;
  }

  String proto = getValue(line, "proto");
  if (proto.length() == 0) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
             ",actual,unknown");
    handshakeOk = false;
    return;
  }

  if (proto.equalsIgnoreCase(PROTO_WIRE)) {
    sendLine("RC:ACK,app,RC");
    handshakeOk = true;
    DBG_PRINTLN("[HS] ACK - session ready");
    return;
  }

  sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
           ",actual," + proto);
  handshakeOk = false;
}

void handleLine(const String& line) {
  int colon = line.indexOf(':');
  if (colon <= 0) return;

  String app = line.substring(0, colon);
  String rest = line.substring(colon + 1);
  int comma = rest.indexOf(',');
  String type = (comma < 0) ? rest : rest.substring(0, comma);

  if (type.equalsIgnoreCase("CONNECT")) {
    handleConnect(app, line);
    return;
  }

  if (!handshakeOk) return;
  if (!app.equalsIgnoreCase("RC")) return;

  if (type.equalsIgnoreCase("CTRL")) {
    g_lx = getValue(line, "lx").toInt();
    g_ly = getValue(line, "ly").toInt();
    g_rx = getValue(line, "rx").toInt();
    g_ry = getValue(line, "ry").toInt();
    g_lk = getValue(line, "lk").toInt();
    g_rk = getValue(line, "rk").toInt();
    String swHex = getValue(line, "sw");
    g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
```

```
503
504     onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505   } else if (type.equalsIgnoreCase("BTN")) {
506     onButton(getValue(line, "id").toInt());
507   }
508 }
```

Anexo B `TelemetryChannels.h`

La cabecera va aparte. El `.ino` la incluye y no la copia.

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  /*
6   * Phone CH1...CH8 are RC:PLOT v0...v7.
7   * Edit kTelemetryChannels[] in the sketch to choose what each channel reports (0-255).
8   *
9   * TEL_VEHICLE  built-in signal (temp, throttle, battery, steer, lights, knobs)
10  * TEL_ADC      analogRead(pin), 0-4095 scaled to 0-255
11  * TEL_GPIO     digitalRead(pin) → 0 or 255 (an output pin reports what was written)
12  * TEL_PWM      LEDC duty on pin. dutyMin maps to 0, dutyMax maps to 255.
13  *              8-bit motor PWM uses 0...255. 12-bit servo travel 1000-2000 µs uses 204...409.
14  *              Attach that pin with ledcAttach / ledcSetup before it is read.
15  */
16
17 enum TelKind : uint8_t {
18   TEL_VEHICLE = 0,
19   TEL_ADC,
20   TEL_GPIO,
21   TEL_PWM,
22 };
23
24 enum TelVehicle : uint8_t {
25   TEL_TEMP = 0,
26   TEL_THROTTLE,
27   TEL_BATT,
28   TEL_STEER,
29   TEL_LIGHTS,
30   TEL_KNOB_L,
31   TEL_KNOB_R,
32   TEL_RY,
33 };
34
35 struct TelChannel {
36   TelKind kind;
37   TelVehicle vehicle;
38   int pin;
39   int ledcChannel;
40   int dutyMin;
41   int dutyMax;
42 };
43
44 using TelVehicleFn = uint8_t (*)(TelVehicle id);
45
46 inline uint8_t telemetryReadPwm(const TelChannel& ch) {
47   if (ch.dutyMax <= ch.dutyMin) return 0;
48   uint32_t duty = 0;
49 #if ESP_ARDUINO_VERSION_MAJOR >= 3
50   if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
51 #else
52   if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
```

```
#endif
  long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
  return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
}

inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle) {
  switch (ch.kind) {
    case TEL_ADC:
      if (ch.pin < 0) return 0;
      return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0, 4095, 0, 255);
    case TEL_GPIO:
      if (ch.pin < 0) return 0;
      return digitalRead(ch.pin) ? 255 : 0;
    case TEL_PWM:
      return telemetryReadPwm(ch);
    default:
      return vehicle ? vehicle(ch.vehicle) : 0;
  }
}

inline void telemetryReadAll(uint8_t out[8], const TelChannel table[8], TelVehicleFn vehicle) {
  for (int i = 0; i < 8; i++) out[i] = telemetryReadOne(table[i], vehicle);
}
```